

Assignment 3B

Window Functions

Approach

I need data that tracks multiple things over time, so I created fake daily stock prices, by use of Claude, for AAPL, MSFT, and GOOG covering about 130 trading days from October 2023 through March 2024. I chose this range because it crosses January 1st, which lets me test whether the year-to-date average resets correctly. I entered the data into my R file so I wouldn't need a separate file or random data generator.

I'll calculate the averages using both dplyr, check that the year-to-date calculations work for each stock and year, that the six-day moving average only appears once six trading days are available, and that the data has the expected rows, dates, and no missing values.

Code Deliverable

```
library(dplyr)
```

```
Attaching package: 'dplyr'
```

```
The following objects are masked from 'package:stats':
```

```
filter, lag
```

```
The following objects are masked from 'package:base':
```

```
intersect, setdiff, setequal, union
```

```
library(tidyr)  
library(readr)  
library(lubridate)
```

```
Attaching package: 'lubridate'
```

The following objects are masked from 'package:base':

date, intersect, setdiff, union

```
library(ggplot2)
library(slides)
```

Warning: package 'slides' was built under R version 4.6.1

```
prices <- read_csv("https://raw.githubusercontent.com/daanishrasheed/DATA607/
refs/heads/main/Assignment%203B/prices.csv", col_types = cols(
  ticker = col_character(),
  date   = col_date(format = "%Y-%m-%d"),
  price  = col_double()
)) |>
  arrange(ticker, date)

glimpse(prices)
```

```
Rows: 387
Columns: 3
$ ticker <chr> "AAPL", "AAPL", "AAPL", "AAPL", "AAPL", "AAPL", "AAPL", "AAPL",...
$ date   <date> 2023-10-02, 2023-10-03, 2023-10-04, 2023-10-05, 2023-10-06, 20...
$ price  <dbl> 170.47, 164.73, 167.86, 165.67, 165.69, 164.75, 163.08, 163.51,...
```

Row counts, date ordering, and missingness all check out before any window function runs on top of this data.

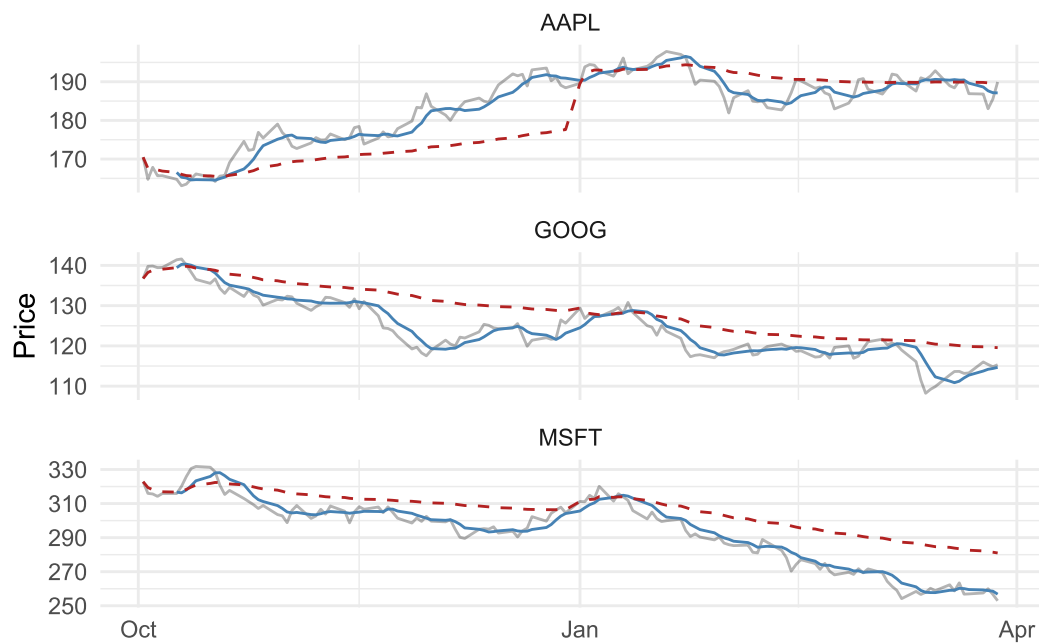
```
pr <- prices |>
  mutate(yr = year(date)) |>
  group_by(ticker, yr) |>
  arrange(date, .by_group = TRUE) |>
  mutate(ytd_avg = cummean(price)) |>
  ungroup() |>
  group_by(ticker) |>
  arrange(date, .by_group = TRUE) |>
  mutate(
    ma_6day = slide_dbl(price, mean, .before = 5, .complete = TRUE)
  ) |>
  select(-yr)
```

```
pr
```

```
# A tibble: 387 × 5
# Groups:   ticker [3]
  ticker date      price ytd_avg ma_6day
  <chr> <date>    <dbl> <dbl> <dbl>
1 AAPL  2023-10-02  170.   170.   NA
2 AAPL  2023-10-03  165.   168.   NA
3 AAPL  2023-10-04  168.   168.   NA
4 AAPL  2023-10-05  166.   167.   NA
5 AAPL  2023-10-06  166.   167.   NA
6 AAPL  2023-10-09  165.   167.  167.
7 AAPL  2023-10-10  163.   166.  165.
8 AAPL  2023-10-11  164.   166.  165.
9 AAPL  2023-10-12  165.   166.  165.
10 AAPL 2023-10-13  166.   166.  165.
# i 377 more rows
```

```
ggplot(pr, aes(x = date)) +
  geom_line(aes(y = price), alpha = 0.3) +
  geom_line(aes(y = ma_6day), color = "steelblue") +
  geom_line(aes(y = ytd_avg), color = "firebrick", linetype = "dashed") +
  facet_wrap(~ticker, ncol = 1, scales = "free_y") +
  labs(
    x = NULL, y = "Price"
  ) +
  theme_minimal()
```

```
Warning: Removed 15 rows containing missing values or values outside the scale
range
(`geom_line()`).
```



Conclusion

The 6-day moving average was much more volatile compared to the year-to-date average. It was highly responsive to any fluctuations in the price over the given week whereas the year-to-date average was slow and smooth, making a jump only during reset at the beginning of the new year, as expected for an average based on many more data points. A small window is noise-sensitive, whereas a large window smooths out the noise, and the difference between the two is obvious when plotted side-by-side.

A few ideas about how this project can be continued include adding another, larger moving average (for example, of 20 days) besides the 6-day one, since comparing two moving averages with different windows side-by-side is often used by traders to identify trends; using not a fixed year-to-date average, but rather a rolling one-year average.